FINGERSCAN VIEWER · DOKUMENTATION

Schwarzer Stift, helle Haut

Automatische Erkennung einer Stiftmarkierung auf dem isolierten Finger und Berechnung der eingekreisten Fläche

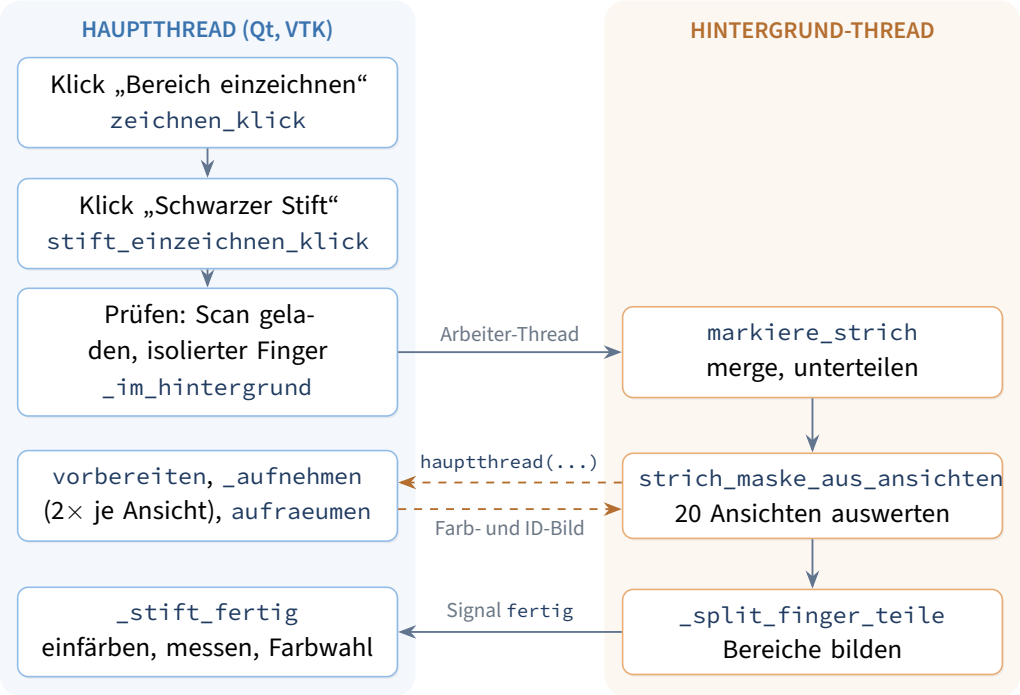
Kurz gesagt Der Viewer fotografiert den isolierten Finger aus 20 Richtungen. Jedes Bild wird mit einer nachgebauten GIMP-Filterkette bearbeitet, die nur den schwarzen Stift übrig lässt. Über ein zweites, unsichtbares „ID-Bild“ wird jedem dunklen Pixel das passende Dreieck (Face) des Meshes zugeordnet. Am Ende wird der Finger an diesen Faces aufgetrennt: Alles, was nicht mit dem Rest des Fingers verbunden ist, liegt innerhalb der Markierung.

Code: `logik/schwarzer_stift.py` Einstellungen: `konstanten.py` Oberfläche: `userinterface.py`, `ui_unterklassen/hauptmenue/oberflaeche.py`

1 Aufruf aus der Oberfläche

1. Isolierten Finger laden (Ordner `isolierte_scans`). Nur dort ist die Funktion erlaubt.
2. Im Hauptmenü „**Bereich einzeichnen**“ klicken → `zeichnen_klick` blendet die Malen-Auswahl ein.
3. Knopf „**Schwarzer Stift, helle Haut**“ klicken → `stift_einzeichnen_klick` (verbunden in `oberflaeche.py`).
4. Während der Suche liegt die Ladeanzeige über dem Viewer, die Bedienung ist gesperrt.
5. Danach sind die gefundenen Bereiche farbig eingefärbt, Fläche und Umfang stehen im Hinweis. Mit „Untersuchung 1–5“ wird die Markierung gespeichert (`farbenwahl`).

Warum zwei Threads? VTK und Qt dürfen nur aus dem Hauptthread benutzt werden. `markiere_strich` bekommt deshalb die Funktion `hauptthread` mitgegeben (`HauptthreadAufruf` aus `hintergrund_arbeit.py`). Sie führt einen Block im Hauptthread aus und wartet auf das Ergebnis. Schneller wird es dadurch nicht, aber die Oberfläche bleibt bedienbar. Ohne Thread ist `hauptthread` einfach ein direkter Aufruf (`_direkt`).



Alles, was den Viewer anfasst, läuft im Hauptthread. Gerechnet wird im Hintergrund, damit sich die Ladeanzeige drehen kann.

2 Wer ruft was auf

Funktion	aufgerufen von	Thread	Aufgabe
`rendere_teile` isolate_finger.py	Laden eines Scans	Haupt	Zeigt den Finger, Name `scan_teil_...` je Mesh
`stift_einzeichnen_klick`	Knopf in der Oberfläche	Haupt	Prüft Voraussetzungen, startet die Suche
`_im_hintergrund`	`stift_einzeichnen_klick`	Haupt	Ladeanzeige, Sperre, startet `Arbeiter`
`markiere_strich`	`Arbeiter.run`	Hintergrund	Gesamter Ablauf, gibt Bereiche und Bericht zurück
`_direkt`	Standardwert von `hauptthread`	wie Aufrufer	Ruft eine Funktion direkt auf
`strich_maske_aus_ansichten`	`markiere_strich`	Hintergrund	Welche Faces sind Strich?
`vorbereiten` / `aufraeumen`	`strich_maske_...` über `hauptthread`	Haupt	ID-Actor anlegen, Fremdes ausblenden, danach alles zurück
`_aufnehmen`	`strich_maske_...` über `hauptthread`	Haupt	Ein Screenshot (Farbe oder ID)
`_kamera_setzen`	`_aufnehmen`	Haupt	Blickrichtung, `reset_camera`
`_eingabe_sperren`	`vorbereiten` / `aufraeumen`	Haupt	Maus im Viewer aus/an
`_kamera_richtungen`	`strich_maske_...`	Hintergrund	20 Blickrichtungen
`_id_farben` / `_farben_zu_ids`	`strich_maske_...`	Hintergrund	Face-Nummer ↔ Farbe
`gimp_kette`	`strich_maske_...`	Hintergrund	Filterkette, nur auf Finger-Pixeln
`_srgb_zu_linear` / `_linear_zu_srgb`	`gimp_kette`	Hintergrund	Gamma hin und zurück
`_pixel_je_mm`	`strich_maske_...`	Hintergrund	Maßstab der Ansicht
`_luecken_im_bild_schliessen`	`strich_maske_...`	Hintergrund	Brücken über kleine Lücken
`_strecke_zeichnen`	`_luecken_im_bild_schliessen`	Hintergrund	Malt eine Brücke ins Bild

Funktion	aufgerufen von	Thread	Aufgabe
`_diagnose_schreiben`	`strich_maske_...`	Hintergrund	Nur wenn Diagnose-Ordner gesetzt
`_split_finger_teile`	`markiere_strich`	Hintergrund	Finger an den Strichen auftrennen
`_flaeche_aus_faces`	niemand	unbenutzt	Überbleibsel einer früheren Version
`_stift_fertig`	Signal `Arbeiter.fertig`	Haupt	Einfärben, `_messwerte_je_bereich`, Hinweis

3 Die GIMP-Kette

Die Kette wurde zuerst in GIMP an einem Screenshot des Viewers ausprobiert und dann in `gimp_kette` nachgebaut. Ein Pixel mit den Kanälen $R, G, B \in [0, 1]$ durchläuft:

1. **Linear machen.** sRGB-Werte sind gammakorrigiert, gerechnet wird linear:

$$c_{\text{lin}} = \begin{cases} c/12{,}92 & c \le 0{,}04045 \\ \left(\dfrac{c + 0{,}055}{1{,}055}\right)^{2{,}4} & \text{sonst} \end{cases}$$

2. **Entsättigen** — GIMP: Sättigung, Skala 0,7

$$Y = 0{,}2126\,R + 0{,}7152\,G + 0{,}0722\,B, \qquad c' = Y + s\,(c - Y), \quad s = 0{,}7$$

3. **Aufhellen** — GIMP: Helligkeit +9 %

$$c'' = c' + h\,(1 - c'), \quad h = 0{,}09$$

4. **Kontrast** um die Mitte spreizen — GIMP: Kontrast +48 %

$$c''' = (c'' - 0{,}5) \cdot \tan\!\left(\tfrac{(k+1)\,\pi}{4}\right) + 0{,}5, \quad k = 0{,}48 \;\Rightarrow\; \text{Steigung} \approx 2{,}31$$

5. **Zurück nach sRGB**, auf $[0, 1]$ begrenzen, dann das **Maximum** der drei Kanäle mal 255.

6. **Schwelle:** Wert < 120 heißt Stift.

> **Warum das Maximum?** Ein Pixel zählt nur dann als Stift, wenn *alle drei* Kanäle dunkel sind. Haut hat immer einen hellen Rotkanal und fällt damit sicher heraus, auch im Schatten.

Die Schwelle 120 gilt nur für Bilder, die genau wie der Viewer aussehen. Deshalb rendert die Erkennung mit derselben Funktion (`rendere_teile`) und derselben Kantenglättung (`msaa`). Gerechnet wird nur auf den Finger-Pixeln, das sind rund 9 % des Bildes.

4 Vom Bild zum Mesh

Zwei Bilder je Ansicht

Für jede der 20 Richtungen (10 rundum, je 5 von $\pm 35°$ schräg) entstehen in einem Rutsch zwei Screenshots mit identischer Kamera:

- **Farbbild**: nur die Fingermeshes, wie im Viewer. Text, Kugeln oder alte Bereiche werden vorher ausgeblendet.
- **ID-Bild**: Face i bekommt die Farbe $i + 1$, verteilt auf drei Bytes, ohne Licht und ohne Kantenglättung. So steht in jedem Pixel, welches Face dort zu sehen ist.

Aufbereitung je Ansicht

- **Lücken schließen**: Glanzlichter unterbrechen den Strich. Liegen zwei Stücke (je ≥ 30 px) näher als 1 mm, wird eine 0,8 mm dicke Brücke gezogen. Den Maßstab in Pixel pro mm liefert `_pixel_je_mm` aus dem ID-Bild.
- **Nur frontale Faces**: Ein Face zählt nur, wenn die Kamera nicht flacher als etwa 70° auf es schaut. Am Umriss wäre die Zuordnung zu unsicher.
- Über alle Ansichten gesammelt ergibt das die Strichmaske: ein Ja/Nein je Face.

5 Bereiche bilden

`_split_finger_teile` trennt den Finger an der Strichmaske auf:

1. Haut ohne Strich in zusammenhängende Gebiete zerlegen.
2. **Außenteil** = das Gebiet mit den meisten offenen Rändern, denn nur dort liegt die Schnittkante an der Fingerbasis.
3. Lose Inseln, die schon ohne Strich nicht mit dem Finger verbunden sind, zählen ebenfalls als außen.
4. Alles andere (Strich, Innenfläche, Taschen, Krümel) wieder in Gebiete zerlegen. Jedes Gebiet mit mindestens 200 Haut-Faces ist ein **Bereich**.
5. Nach Fläche sortiert zurückgeben, Bereich 1 ist der größte.

> **Was zur Fläche gehört** Innenfläche *und* Strich. Löcher im Strich werden automatisch mit aufgefüllt, weil sie keine Verbindung nach außen haben. Einbuchtungen im Strich bleiben erhalten.

6 Alle Funktionen im Detail

6.1 Filterkette

`LUMINANZ = [0.2126, 0.7152, 0.0722]` — Konstante

Gewichte für den Grauwert Y nach Rec. 709. Grün zählt am meisten, weil das Auge dafür am empfindlichsten ist.

`_srgb_zu_linear(c)` — Hintergrund

Aufgerufen von `gimp_kette`

Wandelt sRGB-Werte $0 \ldots 1$ in lineares Licht um (Formel in Abschnitt 3, Schritt 1). Rechnet beide Fälle mit `np.where` auf dem ganzen Array.

`_linear_zu_srgb(c)` — Hintergrund

Aufgerufen von `gimp_kette`

Umkehrung davon. Negative Werte werden vorher auf 0 gesetzt, sonst entstünde bei der Potenz $1/2{,}4$ ein ungültiges Ergebnis.

`gimp_kette(rgb)` — Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`, mit `farbbild[sichtbar]`

Die komplette Filterkette aus Abschnitt 3. Nimmt ein Bild $(H, B, 3)$ oder eine Liste von Pixeln $(N, 3)$ und gibt je Pixel einen Wert $0 \ldots 255$ zurück. Klein heißt dunkel heißt Stift. Die Schwelle wird erst beim Aufrufer angewendet.

6.2 Kamera und Aufnahmen

`_kamera_richtungen()` — Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`

Liefert 20 Einheitsvektoren: einen Ring auf Höhe 0° mit `ANSICHTEN_RUNDUM` Kameras und je einen Ring bei $\pm$`ANSICHT_SCHRAEG_GRAD` mit `ANSICHTEN_SCHRAEG` Kameras. Die schrägen Ringe treffen Kuppe und Basis, wo der Strich sonst nur streifend gesehen würde.

`_id_farben(anzahl)` — Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`, für das ID-Mesh

Gibt Face i die Farbe $i + 1$: unteres Byte in Rot, mittleres in Grün, oberes in Blau. Schwarz (0) bleibt für den Hintergrund frei. Reicht für bis zu 16,7 Millionen Faces.

`_farben_zu_ids(bild)` Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`

Umkehrung: setzt die drei Bytes wieder zu einer Zahl zusammen und zieht 1 ab. Hintergrund ergibt -1.

`_kamera_setzen(plotter, mittelpunkt, richtung)` Haupt

Aufgerufen von `_aufnehmen`

Stellt die Kamera auf `mittelpunkt + richtung` mit Blick zur Fingermitte. „Oben“ ist die Fingerachse, außer die Kamera schaut fast entlang der Achse. Danach rückt `reset_camera` so weit weg, dass der ganze Finger ins Bild passt.

`_aufnehmen(plotter, mittelpunkt, richtung, scan_schauspieler, id_schauspieler, ist_id, hintergrund)` Haupt

Aufgerufen von `strich_maske_aus_ansichten` über `hauptthread`, zweimal je Ansicht

Schaltet nur Sichtbarkeiten um, damit Farb- und ID-Bild Pixel für Pixel aufeinander passen. **Farbbild**: Fingermeshes an, `msaa`, Viewer-Hintergrund. **ID-Bild**: nur das ID-Mesh, keine Kantenglättung, schwarzer Hintergrund. Dann Kamera setzen und Screenshot zurückgeben.

`_eingabe_sperren(plotter, gesperrt)` Haupt

Aufgerufen von `vorbereiten` (sperren) und `aufraeumen` (freigeben)

Schaltet die Maus im VTK-Fenster ab. Ein Klick während der Aufnahme würde die Kamera verstellen und die Bilder unbrauchbar machen.

`_diagnose_schreiben(ordner, nummer, farbbild, dunkel)` Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`, nur wenn `diagnose_ordner` gesetzt ist

Speichert `ansicht_XX_farbe.png` und `ansicht_XX_maske.png` (schwarz = Strich). Der Ordner muss außerhalb des Repositorys liegen, es sind Patientendaten.

6.3 Maßstab und Lücken

`_pixel_je_mm(id_bild, sichtbar, fein, richtung)` Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`, einmal je Ansicht

Teilt die Diagonale der sichtbaren Pixel durch die Diagonale der sichtbaren Faces in mm. Die Punkte werden vorher auf die Bildebene projiziert, die Tiefe in Blickrichtung zählt also nicht mit. Gibt 0 zurück, wenn nichts zu sehen ist.

`_luecken_im_bild_schliessen(dunkel, spanne_px, dicke_px)` Hintergrund

Aufgerufen von `strich_maske_aus_ansichten`, einmal je Ansicht

Wiederholt höchstens `MAX_LUECKEN`-mal: Strichstücke beschriften (8er-Nachbarschaft), nur Stücke ab `MIN_STUECK_PIXEL` behalten, mit einem KD-Baum das nächste Pixelpaar zweier verschiedener Stücke suchen. Ist es höchstens `spanne_px` entfernt, wird eine Brücke gezogen. Schluss, sobald nur noch ein Stück übrig ist oder die nächste Lücke zu groß ist.

`_strecke_zeichnen(bild, von, nach, dicke_px)` Hintergrund

Aufgerufen von `_luecken_im_bild_schliessen`

Malt eine gerade Linie mit rundem Pinsel (Radius `dicke_px`/2) ins Maskenbild. Am Bildrand wird abgeschnitten.

6.4 Ablauf

`_direkt(funktion)` wie Aufrufer

Aufgerufen von Standardwert für `hauptthread` in `markiere_strich` und `strich_maske_aus_ansichten`

Ruft `funktion()` einfach auf. Damit läuft die Erkennung auch ohne Hintergrund-Thread, zum Beispiel in Tests.

`markiere_strich(plotter, teile, diagnose_ordner=None, hauptthread=_direkt)` Hintergrund

Aufgerufen von `Arbeiter.run`, gestartet aus `stift_einzeichnen_klick`

Der Einstieg. Wirft leere Meshes weg, fügt mehrere Teile zu einem zusammen und unterteilt das Mesh `STIFT_UNTERTEILUNGEN`-mal linear, damit ein Dreieck nur wenige Pixel groß ist. Dann Strichmaske holen und Bereiche bilden. Gibt `(bereiche, bericht)` zurück: eine nach Fläche sortierte Liste von Meshes und `gesehen`, `faces`, `strich`, `bereiche`.

`strich_maske_aus_ansichten(plotter, fein, diagnose_ordner=None, hauptthread=_direkt)` Hintergrund

Aufgerufen von `markiere_strich`

Berechnet Face-Normalen und das ID-Mesh, ruft `vorbereiten` auf und geht dann alle 20 Richtungen durch: zwei Bilder aufnehmen, sichtbare Faces bestimmen, Filterkette auf den Finger-Pixeln, Maßstab, Lücken schließen, getroffene Faces nur auf dem Finger, nur frontale übernehmen. Im `finally` immer `aufraeumen`. Gibt die Strichmaske (ein Bool je Face) und einen Bericht zurück.

`vorbereiten()` **Haupt**

Aufgerufen von `strich_maske_aus_ansichten` über `hauptthread`, innere Funktion

Merkt sich Kamera und Hintergrund, sucht die Fingermeshes am Namensanfang `SCAN_TEIL_NAME` (findet es keine, nimmt es wie früher alle), merkt sich alle anderen Actors samt Sichtbarkeit, legt das ID-Mesh an (ohne Licht), blendet die anderen aus und sperrt die Eingabe.

`aufraeumen()` **Haupt**

Aufgerufen von `strich_maske_aus_ansichten` über `hauptthread`, innere Funktion, immer im `finally`

Entfernt das ID-Mesh, zeigt die Fingermeshes, stellt die Sichtbarkeit der anderen Actors, Hintergrund, `msaa` und Kamera wieder her und gibt die Eingabe frei.

6.5 Flächen

`_split_finger_teile(fein_ganz, strich)` **Hintergrund**

Aufgerufen von `markiere_strich`

Ablauf aus Abschnitt 5. `clean()` legt vorher die doppelten Punkte an den UV-Nähten zusammen, sonst zerfiele der Finger in hunderte Stücke. Über das Datenfeld `FaceNr` lassen sich Teilmeshes immer auf das ganze Mesh zurückführen. Gibt eine leere Liste zurück, wenn kein Außenteil gefunden wird oder der Strich nur auf losen Inseln liegt.

`_flaeche_aus_faces(fein, strich)` **unbenutzt**

Aufgerufen von niemand

Baut ein Mesh nur aus den Strich-Faces. Stammt aus einer früheren Version, in der nur der Strich selbst angezeigt wurde. Kann entfernt werden.

7 Die Gegenstücke in der Oberfläche

`stift_einzeichnen_klick(self)` **Haupt**

Aufgerufen von Knopf „Schwarzer Stift, helle Haut“ in `oberflaeche.py`

Prüft, ob ein Scan geladen und es ein isolierter Finger ist, setzt den Zeichenstatus zurück und startet über `_im_hintergrund` die Suche mit `hauptthread=self._hauptthread`.

`_im_hintergrund(self, arbeit, fertig, hinweis)` **Haupt**

Aufgerufen von `stift_einzeichnen_klick` (und Finger isolieren)

Setzt den Hinweis, sperrt die Bedienung, zeigt die Ladeanzeige und startet einen `Arbeiter`-Thread. Wenn er fertig ist, kommt `fertig(ergebnis, fehler)` im Hauptthread.

`_stift_fertig(self, ergebnis, fehler)` **Haupt**

Aufgerufen von Signal `Arbeiter.fertig`

Zeigt Fehler an. Ohne Bereich unterscheidet es zwischen „Strich gefunden, aber nicht geschlossen“ und „Kein Strich gefunden“. Sonst färbt es jeden Bereich in einer eigenen Farbe (`BEREICH_FARBEN`), misst Fläche und Umfang mit `_messwerte_je_bereich` und blendet die Farbwahl zum Speichern ein.

`rendere_teile(plotter, teile)` **Haupt**

Aufgerufen von Laden eines Scans, `isolate_finger.py`

Die eine Stelle, an der der Finger dargestellt wird. Jedes Mesh bekommt den Namen `scan_teil_<Adresse>`, daran erkennt `vorbereiten` die Fingermeshes.

8 Einstellungen

Konstante	Wert	Bedeutung
`ANSICHTEN_RUNDUM` / `_SCHRAEG`	10 / 5	Kameras rundum bzw. je schrägem Ring
`ANSICHT_SCHRAEG_GRAD`	35°	Neigung der schrägen Ringe
`STIFT_SAETTIGUNG`	0,7	GIMP-Kette: Entsättigen
`STIFT_HELLIGKEIT`	0,09	GIMP-Kette: Aufhellen
`STIFT_KONTRAST`	0,48	GIMP-Kette: Kontrast
`STIFT_SCHWELLE_ANSICHT`	120	Darunter ist ein Pixel Stift
`MIN_BLICKWINKEL_COS`	0,35	Etwa 70° zur Normalen
`MAX_LUECKE_MM`	1,0 mm	Größte Lücke, die überbrückt wird
`STRICH_DICKE_MM`	0,8 mm	Dicke einer Brücke
`MIN_STUECK_PIXEL`	30	Kleinere Stücke bekommen keine Brücke
`MAX_LUECKEN`	80	Höchstens so viele Brücken je Ansicht
`STIFT_UNTERTEILUNGEN`	2	Mesh feiner machen, damit die Linie durchgehend getroffen wird
`MIN_FLAECHE_FACES`	200	Mindestgröße für Außenteil und Bereich
`SCAN_TEIL_NAME`	`scan_teil`	Namensanfang der Fingermeshes im Viewer
`ANSICHT_DIAGNOSE_ORDNER`	`None`	Gesetzt: Farbbild und Maske je Ansicht als PNG. Nur außerhalb des Repositorys!

9 Grenzen

- Nur am **isolierten Finger** und nur für **schwarzen Stift auf heller Haut**.
- Der Strich muss **geschlossen** sein. Sonst meldet die Oberfläche „Strich gefunden, er umschließt aber keinen Bereich“.
- Zwei Kreise, deren Striche sich berühren, ergeben einen gemeinsamen Bereich.
- Wird sehr viel Haut fälschlich als Strich erkannt (starker Schatten), kann das falsche Gebiet als Außenteil gewählt werden.
- Nur das größte zusammenhängende Stück des Meshes zählt. Eine Markierung auf einem abgetrennten Teil ginge verloren.
- Ändert sich die Darstellung im Viewer (Licht, Kantenglättung, Hintergrund), muss die Schwelle neu eingemessen werden.